SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

1. *Introduction:*

1.1. Purpose:

This System Analysis Documentation (SAD) presents the technical scope, architectural decisions, and design rationale behind RetOps — an AI-powered demand forecasting and inventory management platform developed by team Hartalis for UMHackathon 2026.

So, the key element of system that has been covered are as followed:

1. ***Architecture**:*
    1. *RetOps is deployed as a serverless web application. The frontend is hosted on Vercel, with a FastAPI backend and SQLite database. The Z AI GLM API serves as the dedicated reasoning service layer.*
2. ***Data Flows:***
    1. *How uploaded transactional and inventory data moves from the user's browser through the backend parsing pipeline, into the GLM inference layer, and back as structured forecast and recommendation outputs.*
3. ***Model Process:***
    1. *The end-to-end workflow of each core feature — from file upload through GLM inference to rendered output — including the validation and self-correction loops.*

4. *Role of Reference:*
    1. *This document serves as the primary technical reference for the development team (Hartalis), QA, and hackathon evaluators assessing the system's architecture and AI integration depth.*

*1.2. **Background**:*

Malaysian SME retailers and food vendors manage inventory manually using paper records, basic spreadsheets, or intuition. While transaction data exists, it is rarely leveraged for forecasting. RetOps addresses this gap by enabling any business owner to upload their existing records and receive GLM-powered demand forecasts and reorder recommendations in plain language.

1. *Previous Version:*
    a. The existing system relies heavily on paper and human workforce. *Inventory decisions are made based on memory, visual shelf checks, and manual counting. There is no systematic analysis of historical sales patterns, seasonal trends, or event-driven demand shifts*
2. *Changes in Major Architectural Components*

- *Replaces manual estimation with data-driven 7-day demand forecasts per product item*
- *Introduces automated reorder recommendations with quantity calculations and plain-language reasoning*
- *Adds contextual awareness via weather API and public holiday calendar integration*
- *Enables scenario modelling through the What-If input feature*
- *Introduces GLM as a structured reasoning layer that interprets inconsistently formatted data without requiring user pre-processing*

1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
Retail / F&B Business Owner	*Primary user uploads transactional and inventory data, reviews demand forecasts and reorder recommendations*	*Clear recommendations in Malay or English, minimal technical knowledge required, trustworthy output with visible reasoning*
Retail Staff / Inventory Manager	*Executes reorder decisions based on GLM-generated shopping list; may also upload data on behalf of the owner*	*Actionable, specific reorder quantities and timing; plain-language explanations they can justify to the owner*
Development Team	*Designs, builds, and maintains the platform across frontend, backend, and GLM integration layers*	*Clear API usage contracts, codebase is modular and well-documented architecture*
QA Team	*Validating the system behavior for all the user roles*	*Defined request/response schemas, clear scope boundaries,*

		documented edge case handling and error states

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	*Serverless web application*
Architecture	*Client-server with stateless REST API backend and GLM as an external dedicated service layer*

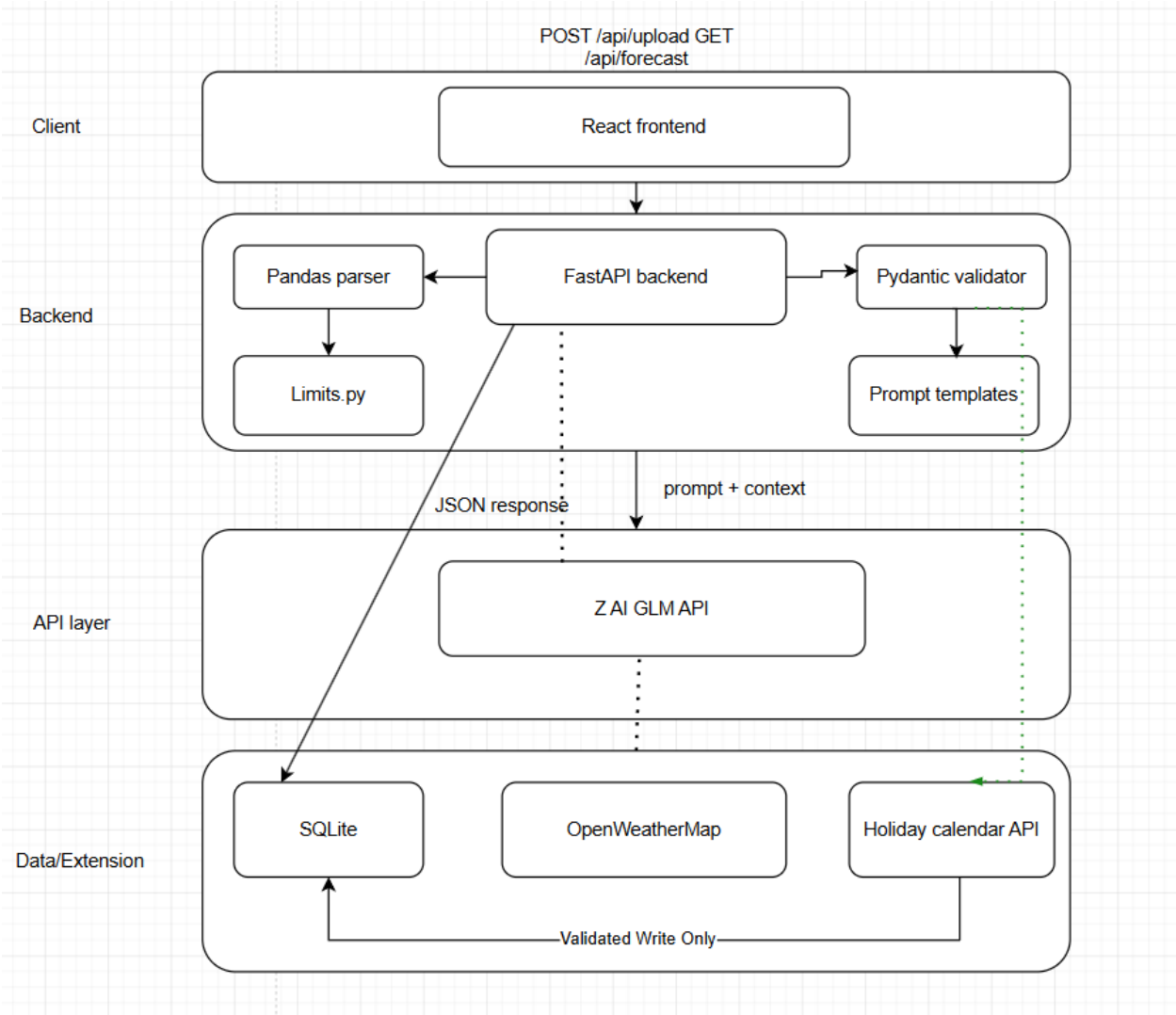
RetOps is structured as a serverless client-server system with two primary user roles: the retail/F&B business owner (primary) and retail staff (secondary). The React frontend communicates with the FastAPI backend via HTTPS REST calls. The backend coordinates three processing pipelines which is data ingestion, GLM inference, and response delivery while the GLM API operates as a fully isolated reasoning service layer with defined input contracts and output schemas.

2.1.2 LLM as Service Layer

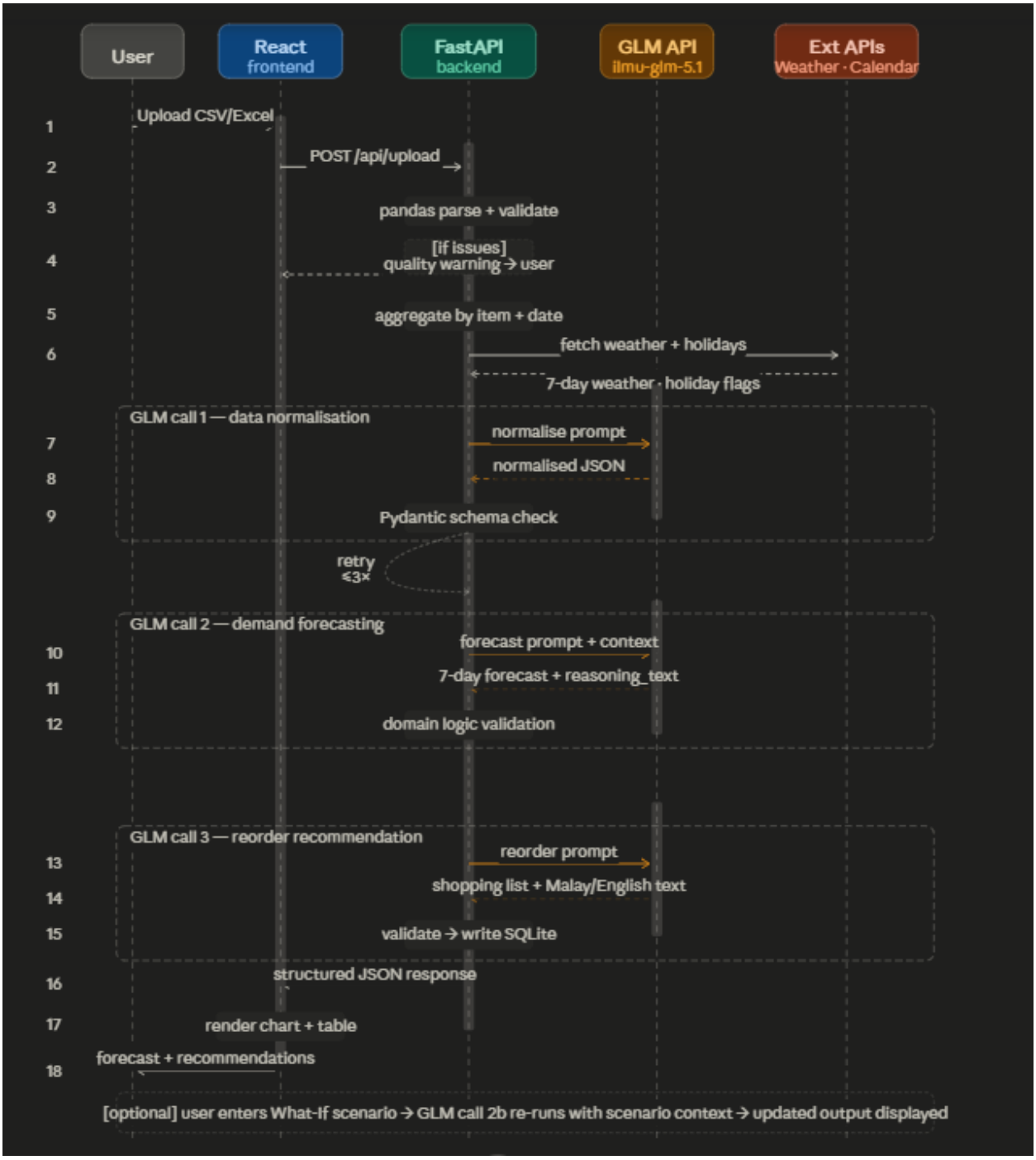
The architecture of your product needs to show how GLM is integrated as a service layer- not limited to a generic "AI Module". The following Dependency diagram includes:

- Prompt Construction: The backend assembles structured prompts using fixed-template system prompts that define persona, constraints, and target JSON schema. User data is injected as a pre-aggregated, token-compressed context payload — never raw file rows.
- Context Window: Each GLM call receives: a system prompt defining behaviour and output schema, a condensed tabular data summary (transactions aggregated by item and date), weather forecast data for the next 7 days, public holiday flags, and any user-provided scenario text. Token budget is capped at 5,000 tokens per request.
- Response Parsing: GLM responses are parsed against Pydantic data models. The validated structure includes: item_name, forecasted_demand (7-value array), confidence_score, reasoning_text, and reorder_flag.
- Token Enforcement: All data is pre-aggregated before reaching GLM. User prompts are capped at 8,000 characters. Payloads exceeding the GLM API context window are rejected with HTTP 422 rather than truncated.
- Self-Correction Loop: On schema or domain logic validation failure, the exact error is injected back into a retry prompt. Maximum 3 retry attempts at temperature=0. Failure after 3 attempts triggers graceful error state.

2.1.3 Dependency Diagram



2.2.2. Sequence Diagram



s2.2. Technological Stack

*2.2.1. Frontend - **React -** Component reuse, fast SPA development, native integration with Chart.js for forecast visualisation*

*2.2.2. Backend - **FastAPI -** Async support for non-blocking GLM calls, fast development cycle, native pandas and Pydantic integration*

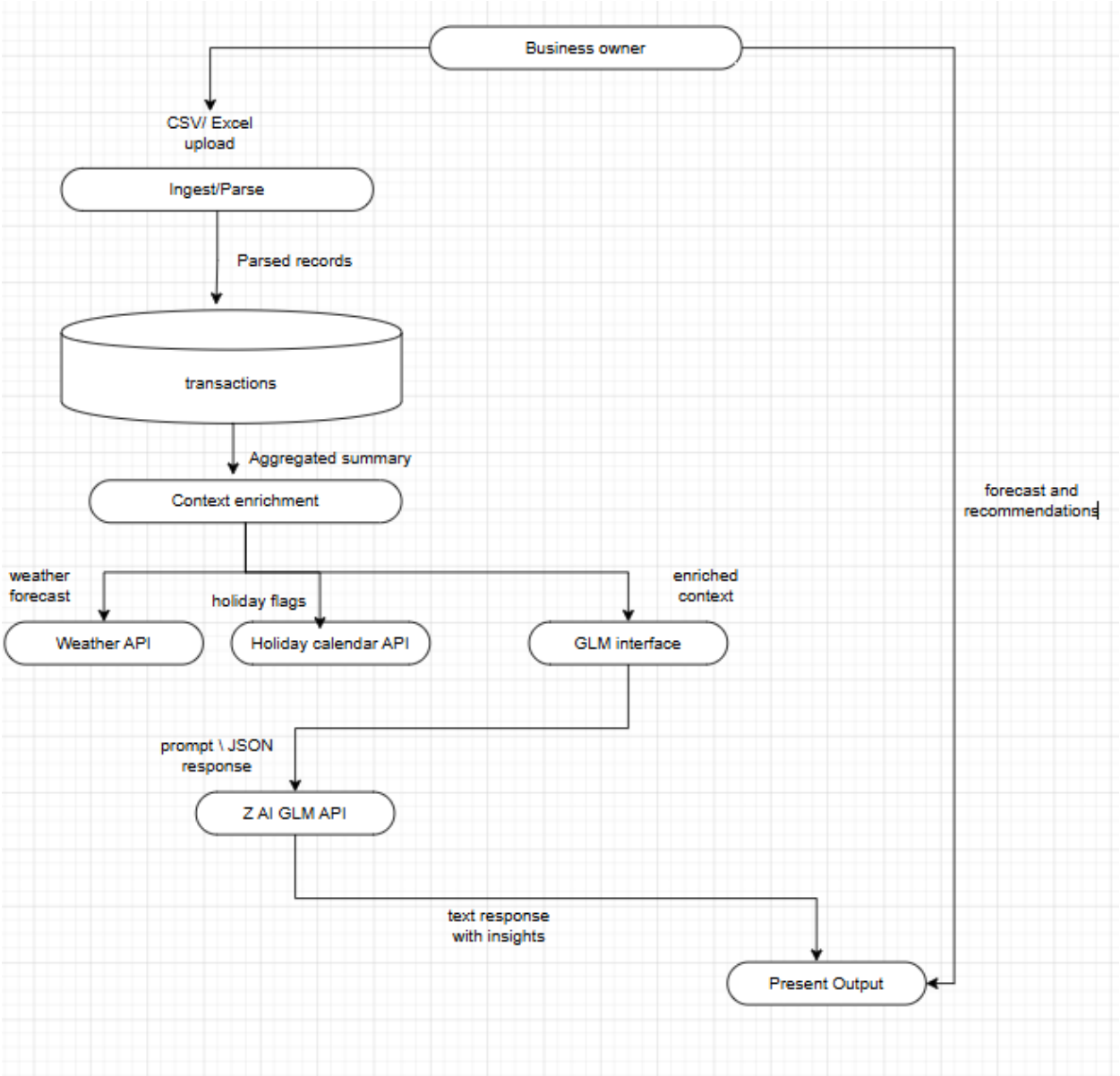
2.2.3. Database - SQLite - Lightweight, zero-configuration setup appropriate for hackathon scope and single-instance deployment

2.2.4. Cloud/Deployment - Vercel - Zero-config serverless deployment, automatic HTTPS, seamless React integration

2.3. Key Data Flows

2.3.1. Here, do show the how data are moving through the system and how its structure and stored

2.3.1.1. Data Flow Diagram (DFD)



2.3.2. **Normalized Database Schema**

The SQLite database is designed to Third Normal Form (3NF) to eliminate redundancy and ensure relational integrity across all entities.

Table	Schema & Notes
sessions	session_id (PK), created_at, status — tracks each upload session lifecycle
uploads	upload_id (PK), session_id (FK → sessions), file_type, file_path, uploaded_at, row_count — records each file upload event
transactions	txn_id (PK), upload_id (FK → uploads), item_name, quantity, unit_price, sale_date — normalised transaction records extracted from uploads. Linked to uploads (not sessions) to support re-uploads within a session without orphaned records.
inventory_items	item_id (PK), upload_id (FK → uploads), item_name, current_stock, unit — normalised inventory records from optional stock upload
forecasts	forecast_id (PK), session_id (FK → sessions), item_name, forecast_date, predicted_demand, confidence_score, reasoning_text — stores per-item GLM forecast outputs. Keyed to session_id to allow What-If variants to coexist with baseline forecast.
recommendations	rec_id (PK), forecast_id (FK → forecasts), item_name, reorder_qty, reorder_by_date, explanation_text, scenario_context — stores GLM reorder outputs. scenario_context records any What-If input that influenced the recommendation.

3NF compliance notes: transactions and inventory_items are keyed to upload_id rather than session_id to eliminate transitive dependency — session metadata is not repeated in each row. forecasts and recommendations reference session_id and forecast_id respectively, resolving the M:N relationship between sessions and product-level predictions through the forecasts junction.

3. Functional Requirements & Scope

3.1. Minimum Viable Product:

3.1.1.

#	Features	Description
1	Inventory Optimisation	Customers browse the menu of different vendors, choose the items as per preferred quantity, add to the card and pay accordingly.
2	Demand forecasting	Visualization, GLM takes last 30 days of sales + day-of-week patterns and predicts next 7 days per menu item. Output must include GLM's reasoning, not just numbers.
3	Reorder Recommendation	GLM tells you what to buy, how much, and when, with reasoning explained in plain Malay/English. Based on forecast vs current stock, GLM outputs a shopping list with quantities and explains why each item is flagged
4	What-If Scenario	Forecast weather, integrate calendar to tell whether its holiday or not
5	Transactional Data Upload & Parsing .	Users upload CSV or Excel sales data. pandas handles format inconsistencies and extracts the last 30 days of records. Data quality summary is returned before processing

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality (e.g.)	***Requirements (e.g.)***	***Implementation (e.g)***
Scalability	*System must handle concurrent sessions during hackathon demo without degradation (demo scope: up to 10 concurrent users)*	*FastAPI async handles concurrent backend requests; SQLite sufficient for demo concurrency*
Reliability	*GLM call failures must not corrupt session data or crash the application*	*Pydantic schema validation gates all GLM responses before database write; 3-attempt retry loop; HTTP 422 graceful error state on exhaustion*
Maintainability	*Prompt templates and pipeline modules must be independently modifiable without affecting other components*	*Prompts stored as separate template files; ingestion, forecasting, and recommendation modules are independently structured; SQLite schema versioned*
Token Latency	*GLM API calls must return within 5 seconds per individual call at p95*	*Timeout set at 8 seconds per GLM call; simplified fallback prompt attempted on timeout; async non-blocking calls where pipeline allows*
Cost Efficiency	*Average token cost per session must not exceed 5,000 tokens*	*Data aggregated before GLM injection; prompt templates use fixed-length structures; repeated prompts cached within session; user input capped at 8,000 characters*

3.3. Out of Scope / Future Enhancements

- *Real-time POS system integration — live data streaming from point-of-sale hardware*
- *Automated purchase order generation with direct supplier API integration*
- *Multi-user workspace with role-based access control and audit trails*
- *Mobile application (iOS / Android native or React Native)*
- *Production-grade authentication with persistent user accounts and data history*
- *Historical trend analysis beyond the 30-day rolling window*
- *In-app notifications and scheduled reorder alerts via email or SMS*
- *Support for file formats beyond CSV and Excel (e.g., JSON exports, PDF invoices)*
- *Migration from SQLite to a production-grade database (e.g., PostgreSQL) for concurrent multi-user access*

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. Grayscale Rollout & A/B Testing:

For post-hackathon deployment, new features — such as updated prompt templates or new scenario types — would be released to 5% of active sessions initially. GLM response quality metrics (schema validation pass rate, user action rate on recommendations) are monitored before gradual increase to 100%. A/B testing would compare prompt strategies: for example, zero-shot vs. few-shot prompting for the recommendation step, with the winning variant determined by recommendation acceptance rate.

4.1.2. Emergency Rollback (ER) & Golden Release:

A golden release version of the application is maintained at all times. Emergency rollback is triggered automatically if: the GLM schema validation failure rate exceeds 10% within a 10-minute window, or the end-to-end pipeline error rate exceeds 5%. The automated deployment pipeline rolls back to the golden release, and the system serves the last successfully cached recommendations during the rollback window. Users see a maintenance notification rather than a broken state.

4.1.3. Priority Matrix:

4.1.3.1.

Priority	Condition	Trigger	Action
P1 — Critical	GLM API unavailable or error rate > 10% in 10-minute window	Automated health check every 60 seconds	Emergency rollback to golden release; user-facing maintenance message; alert development team immediately
P2 — High	Pydantic schema validation failure rate > 5%	Output validation log monitoring on each GLM response	Switch to simplified fallback prompt template; alert development team within 15 minutes

P3 — Medium	*File parse error rate > 15% of upload attempts*	*Upload processing error logs*	*Surface improved user guidance on file format requirements; log for developer review*
P4 — Low	*OpenWeatherMap API unavailable*	*Weather API call failure detection*	*Proceed with forecast generation without weather context; note absence in output to user*

4.2. Monitoring:

4.2.1.

- GLM API response latency and error rate — logged per call in backend; alerting on sustained degradation
- Pydantic schema validation pass/fail rate — logged for every GLM response to track hallucination frequency
- Retry loop trigger rate — tracks how often the self-correction mechanism is invoked, indicating prompt quality
- File upload success rate and pandas parse error frequency — tracked per session to identify common data quality issues
- Token consumption per session — logged to enforce cost budget and detect prompt inefficiencies early
- End-to-end pipeline latency (file upload to recommendation rendered) — primary user experience metric
- OpenWeatherMap API availability — monitored separately; degraded mode activated on failure

4.3. Assumptions:

- Users have stable internet access during file upload and GLM inference no offline mode is supported
- Uploaded transactional data contains a minimum of 7 days of records for a meaningful demand forecast
- Business owners have basic familiarity with uploading a file via a web browser
- The Z AI GLM API (ilmu-glm-5.1) maintains availability and response quality consistent with its published specifications
- OpenWeatherMap free tier provides weather forecast data for Malaysian locations with sufficient granularity for demo use
- Public holiday calendar data for Malaysia is accurate and accessible via the integrated API
- All transactional data uploaded by users is owned by the user and contains no personal customer data (PII)
- SQLite is sufficient for concurrent access at hackathon demo scale (up to 10 simultaneous sessions)

4.4. ***External Dependencies:*** *Do list all the external tools that are applied to your system to function. [e.g. below]*

Tools	*Purpose*	*Risks*
Z AI GLM API (ilmu-glm-5.1)	*Core reasoning engine — data normalisation, demand forecasting, reorder recommendation, and What-If scenario adjustment*	*HIGH: GLM API unavailability or rate limiting causes core product failure. Mitigation: 3-attempt retry loop with simplified fallback prompt; graceful HTTP 422 error state; cached last-successful results served during outage.*
OpenWeatherMap API	*Flags upcoming Malaysian public holidays to automatically adjust demand contextFlags upcoming Malaysian public holidays to automatically adjust demand context*	*LOW: API outage removes automatic holiday detection. Mitigation: user can manually specify holidays via the What-If scenario input*
pandas (Python)	*Parses uploaded CSV and Excel files regardless of format inconsistencies — handles missing headers, mixed data types, and irregular column naming*	*LOW: Well-maintained library. Edge case: severely malformed files may fail parsing. Mitigation: clear user-facing error messages with format guidance and example file download.*
Vercel	*Hosts and deploys the React frontend as a serverless application*	*LOW: Vercel outages are rare and brief. Mitigation: static error page served from CDN edge; no critical data is stored in the frontend*

5. *Project Management & Team Contributions*

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. Project Timeline:

Period	*Phase*	*Deliverables*
Day 1–2	*Planning & Setup*	*PRD and SAD finalised, GitHub repository setup, tech stack confirmed, SQLite schema designed*
Day 3–6	*Data Layer*	*File upload UI (React), pandas parsing module, SQLite schema deployed, data ingestion FastAPI endpoints live, data quality validation logic*
Day 6	*Frontend & Visualisation*	*Forecast Chart.js visualisation, reorder recommendation table, What-If scenario input, Malay/English bilingual output rendering, end-to-end flow connected and tested*
Day 7-9	*GLM Integration*	*GLM API access verified* *GLM prompt pipeline (normalisation + forecast + recommendation), Pydantic schema validation, domain logic validators, retry loop, OpenWeatherMap integration, holiday calendar integration*
Day 9	*Testing & Refinement*	*Edge case testing (malformed files, GLM validation failures, empty datasets, oversized inputs), prompt quality refinement, UI polish and error message review*
Day 10	*Demo Preparation*	*Demo script prepared, test datasets created, Vercel deployment verified, PRD and SAD finalised, presentation rehearsal*

5.2. Team Members Role:

Member	*Role*	*Responsibilities*
Chuah Chen Xuan	*Frontend Lead*	*React application architecture, file upload UI, forecast visualisation (Chart.js), What-If scenario interface, Code integration*

Ng Kai Kit	*Frontend Dev, Backend Dev, Database engineer*	*Reorder recommendation table UI, responsive design, error state UI, UX flow refinement, Creating database for RetOps, Code integration*
Chow Chun Seng	*Backend Dev*	*GLM prompt engineering for demand forecasting, Pydantic schema design, domain logic validation*
Muhammad Nafique	*Backend Dev*	*Inventory optimisation module, pandas file parsing pipeline, data ingestion FastAPI endpoints, SQLite schema implementation*
Ethan Tan Zheng Nan	*Backend Dev*	*Reorder recommendation module, context enrichment pipeline (data aggregation + weather + holidays), GLM retry loop and fallback logic, limits.py token/character enforcement*

5.3. Recommendations:

- Migrate SQLite to PostgreSQL for any post-hackathon production deployment to support concurrent multi-user access and transaction isolation.
- Add a user feedback mechanism, a simple "Used this recommendation" or "Ignored this" toggle per shopping list item. This data can be used to evaluate prompt quality and refine the recommendation strategy post-launch.
- For production, replace OpenWeatherMap free tier with a paid plan or alternative API to remove the rate-limiting risk that would affect forecast quality at scale.

6.0 Reference

1. Demizu, T., Fukazawa, Y., & Morita, H. (2023). Inventory management of new products in retailers using model-based deep reinforcement learning. Expert Systems with Applications, 229, 120256. sciencedirect. https://doi.org/10.1016/j.eswa.2023.120256
2. Damron, T., Rupp, W. T., & Smith, A. D. (2016). Inventory control in the retail sector: case studies of best business practices. International Journal of Procurement Management, 9(3), 354. https://doi.org/10.1504/ijpm.2016.076309
3. Luther, D. (2020, September 16). Your Guide to Retail Store Inventory. Oracle NetSuite. https://www.netsuite.com/portal/resource/articles/inventory-management/retail-inventory-management.shtml
4. Malaysia Public Holidays - PublicHolidays.com.my. (2011). PublicHolidays.com.my. https://publicholidays.com.my/
5. Department of Statistics Malaysia. (2025). Wholesale & Retail Trade by Group (3 digit) | OpenDOSM. Dosm.gov.my; OpenDOSM. https://open.dosm.gov.my/data-catalogue/iowrt_3d